
REPORTE DE PRÁCTICA

NFC/RFID COM + APP

Tecnologías Inalámbricas

Fecha: 15/04/2026

Integrantes:

- Eduardo Cadengo López
- Itzel Citlalli Martell De La Cruz
- Damian Alexander Diaz Piña

Índice

1. Datos Generales	3
2. Descripción de la Tecnología RFID/NFC.....	3
3. Usos Actuales de la Tecnología	3
4. Funcionamiento y Descripción del Módulo PN532	4
Principio de operación	4
Características principales.....	4
5. Diagrama de Conexión (ESP32 – PN532 por I2C).....	4
Tabla de pines	4
Diagrama esquemático (ASCII)	5
6. Código Arduino IDE — Funcionamiento y Salidas Numéricas.....	6
Funciones de conversión numérica (líneas clave)	6
Código completo (Arduino IDE)	6
7. Código Python — Lectura del Puerto COM	12
Aspectos clave del código	12
Código Python (lector_com.py).....	12
8. Conclusiones	17
9. Referencias y Enlaces	17

1. Datos Generales

Práctica: Lectura de UID y validación de acceso con tarjeta NFC

Plataforma: ESP32

Módulo sensor: NFC PN532 (comunicación I2C)

Entorno: Arduino IDE + librería Adafruit_PN532

Lenguaje externo: Python 3 (pyserial) — lectura de puerto COM

Objetivo: Leer el UID de una tarjeta NFC/RFID y usarlo como validador para simular el inicio o término de operaciones (apertura de cerraduras, encendido de maquinaria).

2. Descripción de la Tecnología RFID/NFC

RFID (Radio Frequency Identification) es una tecnología de identificación inalámbrica que utiliza radiofrecuencia para leer o escribir información en una etiqueta (tag) mediante un lector. En alta frecuencia (HF), opera a 13.56 MHz y se usa ampliamente para tarjetas de proximidad y sistemas de identificación.

NFC (Near Field Communication) puede considerarse una variante especializada de RFID HF, enfocada a distancias muy cortas (menor a 10 cm). Trabaja sobre estándares como ISO/IEC 14443 (tarjetas de proximidad), donde el lector genera el campo electromagnético y la tarjeta pasiva responde con su identificador único (UID).

Las tarjetas NFC/RFID almacenan un número de serie único (UID) de 4 o 7 bytes. Este UID es la base para los sistemas de control de acceso: el lector lo captura y un microcontrolador lo compara contra una lista autorizada para conceder o denegar el acceso.

3. Usos Actuales de la Tecnología

- Control de acceso a edificios, laboratorios u oficinas mediante credenciales personales.
 - Pagos sin contacto y validación rápida en puntos de venta (tarjetas bancarias NFC).
 - Tarjetas de transporte público: validación de entrada y salida en autobuses y metros.
 - Identificación, trazabilidad e inventarios en bibliotecas, almacenes y logística.
 - Inicio y paro de maquinaria industrial: solo operadores autorizados pueden arrancar equipos.
 - Autenticación en dispositivos móviles y llaves de hotel inteligentes.
-

4. Funcionamiento y Descripción del Módulo PN532

El PN532 es un controlador NFC integrado de NXP Semiconductors, diseñado para comunicación sin contacto a 13.56 MHz. Soporta los modos de lector/escritor para tarjetas ISO/IEC 14443A (MIFARE Classic, MIFARE Ultralight, NTAG) y puede comunicarse con el microcontrolador mediante tres interfaces: I2C, SPI o UART. En esta práctica se emplea el modo I2C (SDA/SCL).

Principio de operación

1. **Campo RF:** el módulo genera un campo electromagnético a 13.56 MHz.
2. **Energización:** cuando una tarjeta pasiva entra al campo, se energiza por inducción.
3. **Detección/Selección:** el PN532 realiza el protocolo ISO14443A (REQA - ATQA - anticollision - SELECT).
4. **Entrega del UID:** el número único de la tarjeta se transfiere al ESP32 para su validación.

Características principales

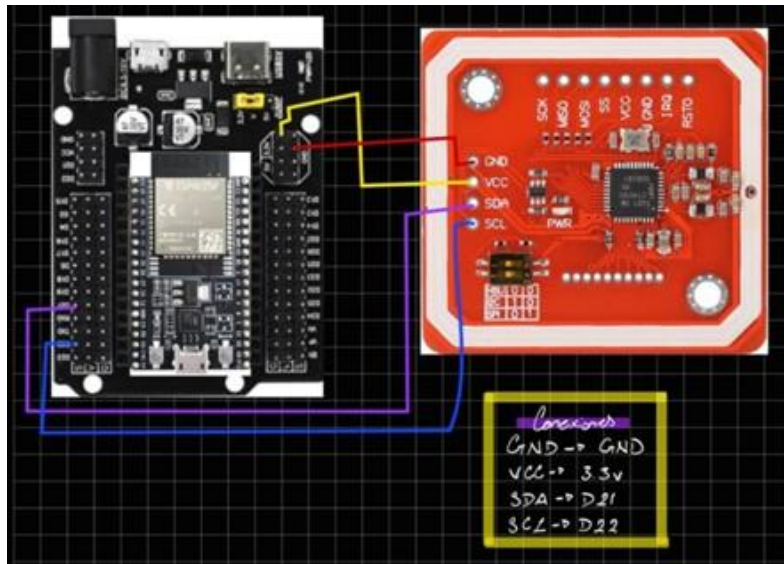
- Voltaje de operación: 3.3 V (algunos módulos incluyen regulador a 5 V).
- Frecuencia: 13.56 MHz (HF).
- Interfaces: I2C (esta práctica), SPI, UART/HSU.
- Soporta tarjetas ISO14443A/B y FeliCa; también modo Peer-to-Peer (NFC Forum).
- Librería Arduino: Adafruit_PN532 (disponible en el Gestor de Librerías del IDE).

5. Diagrama de Conexión (ESP32 – PN532 por I2C)

Tabla de pines

ESP32	PN532
3V3	VCC (3.3 V)
GND	GND
GPIO21 (SDA)	SDA
GPIO22 (SCL)	SCL

Diagrama esquemático (ASCII)



Nota: El GPIO2 se usa como salida de indicación (LED integrado en la mayoría de placas ESP32). Al detectar una tarjeta autorizada se activa HIGH durante 2 segundos (simulando apertura de cerradura o encendido de maquinaria).

6. Código Arduino IDE — Funcionamiento y Salidas Numéricas

El programa integra todas las funcionalidades requeridas: lectura del UID, presentación del código en tres bases numéricas, verificación contra una lista blanca de tarjetas autorizadas y emisión de los datos en formato JSON por el puerto serial para que un programa externo (Python/Web) los consuma.

Funciones de conversión numérica (líneas clave)

uid_a_binario() — Extrae cada bit del byte usando desplazamiento a la derecha (`>> bit`) e imprime 1 ó 0. Cada byte genera 8 bits; los bytes se separan con espacio. Salida: Base 2.

uid_a_decimal() — Acumula los bytes en un entero largo: `valor = (valor << 8) | uid[i]`. El byte 0 es el más significativo. Imprime el número con `Serial.println(valor)`. Salida: Base 10.

uid_a_hexadecimal() — Imprime cada byte con `Serial.print(uid[i], HEX)`; antepone "0" si el valor es menor que 0x10 para mantener dos dígitos por byte. Separa con ":". Salida: Base 16.

enviar_json_serial() — Construye las tres representaciones como cadenas y las encapsula en un objeto JSON con campos `hex`, `dec`, `bin` y `acceso`. El salto de línea final actúa como delimitador de trama para `readline()` en Python.

Código completo (Arduino IDE)

```
#include <Wire.h>           // Biblioteca I2C nativa de Arduino/ESP32
#include <Adafruit_PN532.h> // Biblioteca del módulo PN532 de Adafruit

// PINES I2C para ESP32
// SDA → GPIO 21 | SCL → GPIO 22

#define SDA_PIN 21
#define SCL_PIN 22

// Pin de salida: LED o relé que indica acceso concedido

#define PIN_SALIDA 2          // GPIO2 = LED integrado en la mayoría de ESP32

// Tiempo en ms que permanece activa la salida al leer tarjeta válida
#define TIEMPO_ACTIVACION 2000

// Instancia del objeto PN532 usando I2C
// Los parámetros IRQ y RESET se pasan como -1 si no se usan
Adafruit_PN532 nfc(SDA_PIN, SCL_PIN);

// TARJETAS AUTORIZADAS
```

```

byte tarjetasAutorizadas[][7] = {
    {0x49, 0x1C, 0x33, 0x07, 0x00, 0x00, 0x00}, // Tarjeta 1
    {0x9D, 0x97, 0x24, 0x07, 0x00, 0x00, 0x00} // Tarjeta 2
};

byte numTarjetasAutorizadas = 2;

// Convierte un arreglo de bytes a cadena en BASE 2
// Cada byte se imprime como 8 bits con ceros a la izquierda
void uid_a_binario(uint8_t *uid, uint8_t longitud) {
    Serial.print(" Base 2 (BIN): ");
    for (uint8_t i = 0; i < longitud; i++) {
        for (int bit = 7; bit >= 0; bit--) {
            Serial.print((uid[i] >> bit) & 1); // Extrae bit a bit de mayor a menor
            peso
        }
        if (i < longitud - 1) Serial.print(" "); // Espacio entre bytes para
        legibilidad
    }
    Serial.println();
}

// Función: uid_a_decimal
// Convierte el UID completo a un número entero largo (Base 10)
// Los bytes se concatenan: byte0 es el más significativo
void uid_a_decimal(uint8_t *uid, uint8_t longitud) {
    unsigned long valor = 0;
    for (uint8_t i = 0; i < longitud; i++) {
        valor = (valor << 8) | uid[i]; // Desplazamiento: cada byte ocupa 8 bits
    }
    Serial.print(" Base 10 (DEC): ");
    Serial.println(valor); // Imprime el número en base decimal
}

// Función: uid_a_hexadecimal
// Imprime el UID byte a byte en Base 16 con formato 0xXX
void uid_a_hexadecimal(uint8_t *uid, uint8_t longitud) {
    Serial.print(" Base 16 (HEX): ");
    for (uint8_t i = 0; i < longitud; i++) {
        if (uid[i] < 0x10) Serial.print("0"); // Cero a la izquierda para valores <
        16
        Serial.print(uid[i], HEX); // Imprime el byte en hexadecimal
        if (i < longitud - 1) Serial.print(":"); // Separador estilo MAC address
    }
    Serial.println();
}

```

```
// Función: verificar_acceso
// Compara el UID leído contra la lista de autorizados
// Retorna true si hay coincidencia, false en caso contrario
bool verificar_acceso(uint8_t *uid, uint8_t longitud) {
    for (byte t = 0; t < numTarjetasAutorizadas; t++) {
        bool coincide = true;
        for (byte b = 0; b < longitud; b++) {
            if (uid[b] != tarjetasAutorizadas[t][b]) { // Compara byte a byte
                coincide = false;
                break;
            }
        }
        if (coincide) return true; // UID encontrado en la lista
    }
    return false; // No coincide con ninguna tarjeta registrada
}

// Envía los datos de la tarjeta en formato JSON al puerto COM
// Permite que aplicaciones externas (Python/Web) los lean
void enviar_json_serial(uint8_t *uid, uint8_t longitud, bool acceso) {

    // -- Construir HEX como string --
    String hexStr = "";
    for (uint8_t i = 0; i < longitud; i++) {
        if (uid[i] < 0x10) hexStr += "0";
        hexStr += String(uid[i], HEX);
        if (i < longitud - 1) hexStr += ":";
    }
    hexStr.toUpperCase(); // Letras mayúsculas para estandarizar

    // -- Construir DEC como número largo --
    unsigned long decVal = 0;
    for (uint8_t i = 0; i < longitud; i++) {
        decVal = (decVal << 8) | uid[i];
    }

    // -- Construir BIN como string --
    String binStr = "";
    for (uint8_t i = 0; i < longitud; i++) {
        for (int bit = 7; bit >= 0; bit--) {
            binStr += String((uid[i] >> bit) & 1);
        }
        if (i < longitud - 1) binStr += " ";
    }
}
```



```
// -- Emitir JSON por Serial (puerto COM) --
// Formato: {"hex":"XX:XX:XX:XX","dec":123456,"bin":"...","acceso":true}
Serial.print("{\"hex\\\":\\\"");
Serial.print(hexStr);
Serial.print(\"\\\",\\\"dec\\\":\");
Serial.print(decVal);
Serial.print(\"\\\",\\\"bin\\\":\\\"");
Serial.print(binStr);
Serial.print(\"\\\",\\\"acceso\\\":\");
Serial.print(acceso ? "true" : "false");
Serial.println("}"); // Salto de línea es el delimitador de trama
}

// SETUP – Se ejecuta una sola vez al encender la ESP32
void setup() {
    // Inicia comunicación serial a 115200 baudios
    // Este valor debe coincidir en el lector Python y el Monitor Serial
    Serial.begin(115200);
    delay(100);

    // Configura el pin de salida (LED/relé)
    pinMode(PIN_SALIDA, OUTPUT);
    digitalWrite(PIN_SALIDA, LOW);

    Serial.println(" SISTEMA DE CONTROL DE ACCESO NFC/RFID");
    Serial.println(" ESP32 + Modulo PN532 (I2C)");
    // Inicia el módulo PN532
    nfc.begin();

    // Obtiene la versión del firmware del PN532
    uint32_t versiondata = nfc.getFirmwareVersion();
    if (!versiondata) {
        // Si no responde el módulo, se detiene el programa
        Serial.println("[ERROR] Modulo PN532 no detectado. Revisa conexiones.");
        while (1); // Bucle infinito = detiene la ejecución
    }

    // Imprime datos del firmware para verificar la conexión correcta
    Serial.print("[OK] PN532 detectado. Chip: ");
    Serial.print((versiondata >> 24) & 0xFF, HEX);
    Serial.print(" Firmware: ");
    Serial.print((versiondata >> 16) & 0xFF, DEC);
    Serial.print(".");
    Serial.println((versiondata >> 8) & 0xFF, DEC);
}
```

```
// Configura el PN532 para leer tarjetas ISO14443A (RFID/NFC Tipo A)
nfc.SAMConfig();

Serial.println("\n[LISTO] Acerca una tarjeta NFC o RFID al lector...\n");
}

// LOOP - Se ejecuta continuamente
void loop() {
    uint8_t uid[7];          // Buffer para almacenar el UID (máx. 7 bytes para
                             // NTAG/Mifare)
    uint8_t uidLength;       // Longitud real del UID leído (4 bytes = RFID, 7 bytes =
                             // NFC)
    uint8_t atqa[2];         // Answer To Request Type A (identificador de tipo de
                             // tarjeta)
    uint8_t sak;              // Select Acknowledge (informa el tipo de tarjeta)

    // Espera activa hasta detectar una tarjeta ISO14443A
    // Tiempo de espera (timeout): 0 = no bloquea, 100 = 100 ms
    boolean success = nfc.readPassiveTargetID(PN532_MIFARE_ISO14443A,
                                              uid,
                                              &uidLength);

    if (success) {

        Serial.println("[TARJETA DETECTADA]");
        Serial.print("  Longitud UID : ");
        Serial.print(uidLength, DEC);
        Serial.println(" bytes");

        // --- Mostrar UID en las tres bases numéricas ---
        uid_a_hexadecimal(uid, uidLength); // Base 16
        uid_a_decimal(uid, uidLength);     // Base 10
        uid_a_binario(uid, uidLength);     // Base 2

        // --- Verificar si la tarjeta tiene acceso ---
        bool accesoConcedido = verificar_acceso(uid, uidLength);

        if (accesoConcedido) {
            Serial.println(" [ACCESO CONCEDIDO] Tarjeta autorizada.");
            digitalWrite(PIN_SALIDA, HIGH);          // Activa salida (abre
            cerradura/LED verde)
            delay(TIEMPO_ACTIVACION);                // Mantiene la salida activa
            digitalWrite(PIN_SALIDA, LOW);           // Desactiva la salida
        } else {
```

```
    Serial.println(" [ACCESO DENEGADO] Tarjeta no registrada.");
    // Aquí puedes agregar un buzzer o LED rojo para indicar acceso denegado
}

// --- Enviar datos en formato JSON para el lector Python/Web ---
enviar_json_serial(uid, uidLength, accesoConcedido);

Serial.println("-----\n");

// Espera a que la tarjeta se retire antes de volver a leer
delay(1500);
}
```

7. Código Python — Lectura del Puerto COM

El archivo lector_com.py abre el puerto serial asignado a la ESP32, lee cada línea, la decodifica como JSON y la muestra en consola con colores ANSI. Incorpora reconexión automática ante desconexiones y un mutex (Lock) para acceso seguro desde múltiples hilos, permitiendo integración futura con un servidor Flask.

Aspectos clave del código

- **COM_PORT / BAUD_RATE:**
Variables de configuración al inicio del archivo. BAUD_RATE debe coincidir exactamente con Serial.begin() en el ESP32 (115200).
- **parsear_linea():**
Descarta líneas que no empiezan con '{' (mensajes de debug del ESP32) y decodifica el JSON.
- **serial.Serial(..., timeout=1):**
El timeout de 1 s evita que readline() bloquee indefinidamente; el bucle continúa si no llegan datos.
- **Reconexión automática:**
El bloque try/except captura SerialException y espera 3 segundos antes de reintentar, tolerando reconexiones de la ESP32.

Código Python (lector_com.py)

```
"""
=====
LECTOR DE PUERTO COM - SISTEMA NFC/RFID
Archivo: lector_com.py

Descripción:
    Lee datos JSON enviados por la ESP32 a través del
    puerto serial COM y los muestra en consola.
    Compatible con el servidor Flask (app.py) que
    expone los datos a la interfaz web de Laragon.

Dependencias:
    pip install pyserial

Uso:
    python lector_com.py
    python lector_com.py --port COM5 --baud 115200
=====
"""

import serial          # Comunicación con el puerto serial (COM)
import json            # Parseo de tramas JSON enviadas por la ESP32
```

```

import time            # Control de tiempos y reintentos
import argparse        # Argumentos de línea de comandos
import sys
import threading       # Hilo paralelo para lectura no bloqueante
from datetime import datetime

# -----
# CONFIGURACIÓN POR DEFECTO
# Cambia COM_PORT al puerto que asignó Windows a tu ESP32
# Puedes verlo en: Administrador de dispositivos → Puertos (COM y LPT)
# -----
COM_PORT = "COM4"      # Puerto serial asignado a la ESP32
BAUD_RATE = 115200     # Velocidad en baudios – debe coincidir con el código
                        # Arduino

# -----
# Variable global compartida con el servidor Flask
# Almacena la última lectura recibida de la ESP32
# -----
ultima_lectura = {
    "hex": "--",
    "dec": "--",
    "bin": "--",
    "acceso": None,
    "timestamp": "--"
}

lock = threading.Lock() # Mutex para acceso seguro desde múltiples hilos

def parsear_linea(linea: str) -> dict | None:
    linea = linea.strip()
    if not linea.startswith("{"):
        return None
    try:
        datos = json.loads(linea)

        # --- LÓGICA ADICIONAL EN PYTHON ---
        # Si el HEX coincide con tu tarjeta, forzamos el acceso a True
        if datos.get("hex") == "49:1C:33:07":
            datos["acceso"] = True

        # -----

        return datos
    except json.JSONDecodeError:
        return None

```

```

def formatear_tarjeta(datos: dict) -> str:
    """
    Genera una representación en texto de los datos de la tarjeta
    para mostrarlos en la consola con colores ANSI.
    """

    acceso = datos.get("acceso", False)
    estado = "\033[92m[ACCESO CONCEDIDO]\033[0m" if acceso else "\033[91m[ACCESO
DENEGADO]\033[0m"
    ts      = datos.get("timestamp", "")

    return (
        f"\n{'='*55}\n"
        f"  📇  TARJETA DETECTADA  -  {ts}\n"
        f"{'-'*55}\n"
        f"  HEX (Base 16): {datos.get('hex', '--')}\n"
        f"  DEC (Base 10): {datos.get('dec', '--')}\n"
        f"  BIN (Base  2): {datos.get('bin', '--')}\n"
        f"  Estado       : {estado}\n"
        f"{'='*55}\n"
    )

def leer_serial(port: str, baud: int, callback=None):
    """
    Función principal: abre el puerto serial y lee en bucle.

    Parámetros:
        port      - Nombre del puerto COM (ej. "COM4", "/dev/ttyUSB0")
        baud      - Velocidad en baudios (debe coincidir con la ESP32)
        callback  - Función opcional a llamar con cada tarjeta leída

    El bucle maneja automáticamente desconexiones y reconexiones.
    """

    global ultima_lectura

    while True:
        try:
            print(f"\n[INFO] Intentando conectar a {port} a {baud} baudios...")

            # Abre el puerto serial con timeout de 1 segundo por línea
            # timeout=1 evita que readline() bloquee indefinidamente
            ser = serial.Serial(
                port=port,

```

```
        baudrate=baud,
        bytesize=serial.EIGHTBITS,      # 8 bits de datos por trama
        parity=serial.PARITY_NONE,      # Sin bit de paridad
        stopbits=serial.STOPBITS_ONE,   # 1 bit de parada
        timeout=1                        # Timeout por línea en segundos
    )

    print(f"[OK] Conectado a {port}. Esperando tarjetas NFC/RFID...\n")
    print("      (Presiona Ctrl+C para detener)\n")

    while True:
        # Lee una línea completa hasta el salto de línea (\n)
        # La ESP32 termina cada trama JSON con Serial.println()
        raw = ser.readline()

        if not raw:
            continue    # Timeout de 1 s sin datos → sigue esperando

        try:
            # Decodifica bytes a string usando UTF-8
            linea = raw.decode("utf-8", errors="replace")
        except Exception:
            continue

        datos = parsear_linea(linea)

        if datos is None:
            # Es un mensaje de texto plano de la ESP32 (debug)
            print(f"[ESP32] {linea.strip()}")
            continue

        # Agrega marca de tiempo local a los datos recibidos
        datos["timestamp"] = datetime.now().strftime("%Y-%m-%d %H:%M:%S")

        # Actualiza la variable global de forma thread-safe
        with lock:
            ultima_lectura = datos

        # Muestra los datos en consola
        print(formatear_tarjeta(datos))

        # Llama al callback si existe (lo usa el servidor Flask)
        if callback:
            callback(datos)
```

```

        except serial.SerialException as e:
            print(f"\n[ERROR] Puerto serial: {e}")
            print(f"[INFO] Reintentando en 3 segundos...")
            time.sleep(3) # Espera antes de reconectar (útil si la ESP32 se
desconecta)

        except KeyboardInterrupt:
            print("\n[INFO] Lectura detenida por el usuario.")
            sys.exit(0)

def obtener_ultima_lectura() -> dict:
    """
    Retorna una copia de la última lectura recibida.
    Usada por el servidor Flask para responder peticiones HTTP.
    """
    with lock:
        return dict(ultima_lectura)

# -----
# PUNTO DE ENTRADA – se ejecuta si se corre directamente
# -----
if __name__ == "__main__":
    parser = argparse.ArgumentParser(
        description="Lector de tarjetas NFC/RFID por puerto COM"
    )
    parser.add_argument(
        "--port", default=COM_PORT,
        help=f"Puerto COM (default: {COM_PORT})"
    )
    parser.add_argument(
        "--baud", type=int, default=BAUD_RATE,
        help=f"Baudios (default: {BAUD_RATE})"
    )
    args = parser.parse_args()

    print("=====")
    print("  LECTOR NFC/RFID – Puerto COM → Consola  ")
    print("=====")

    # Inicia la lectura en modo standalone (sin Flask)
    leer_serial(args.port, args.baud)

```

8. Conclusiones

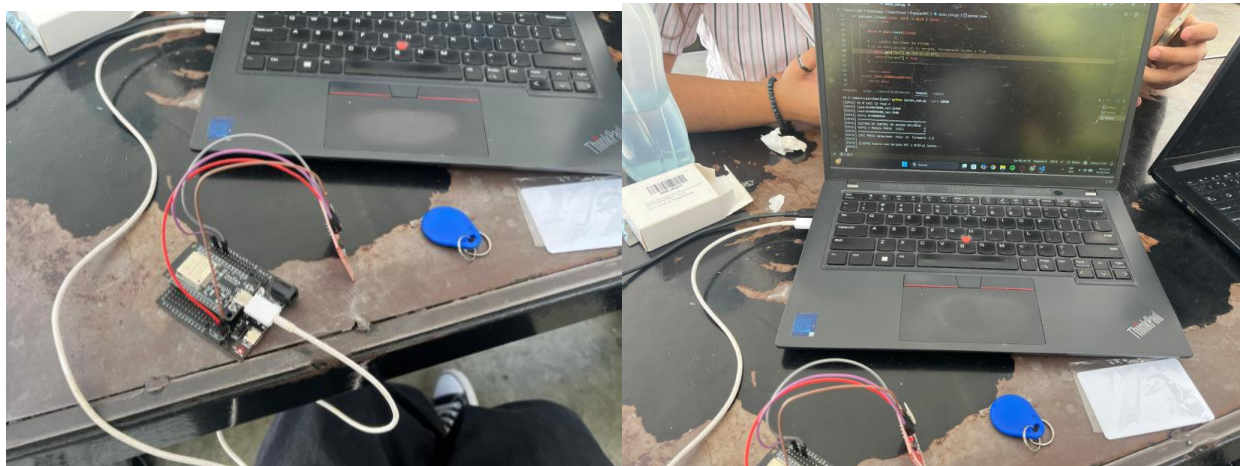
Eduardo: Durante el desarrollo de esta práctica se logró comprender de manera clara cómo funciona la tecnología NFC/RFID aplicada a sistemas reales de control de acceso. La integración entre la ESP32 y el módulo PN532 permitió obtener el UID de las tarjetas de forma confiable y procesarlo en distintos formatos numéricos. Además, el envío de información mediante el puerto COM y su lectura desde Python demuestra que este tipo de sistemas puede escalar fácilmente a aplicaciones más complejas, como plataformas web o sistemas automatizados de seguridad.

Itzel: La práctica permitió observar cómo una tecnología ampliamente utilizada en la vida diaria puede ser implementada desde cero a nivel de hardware y software. El uso del PN532 junto con la ESP32 facilitó la lectura y validación de tarjetas NFC, lo cual ayuda a entender mejor los principios de identificación inalámbrica. La visualización del UID en diferentes bases numéricas refuerza conceptos importantes de representación de datos y demuestra la flexibilidad del microcontrolador para adaptarse a distintos usos.

Damian: A través de esta práctica se confirmó que los sistemas de control de acceso basados en NFC son una solución eficiente y relativamente sencilla de implementar. El correcto funcionamiento del sistema, desde la lectura del UID hasta la validación y activación de una salida, demuestra la confiabilidad del conjunto ESP32–PN532. Además, la comunicación con un programa externo mediante el puerto serial abre la posibilidad de integrar este proyecto con otros sistemas, como bases de datos o interfaces gráficas, ampliando sus aplicaciones prácticas.

9. Videos, GitHub y Evidencias

Evidencias del proyecto:



GitHub del proyecto:

[https://github.com/AresGodKiller/Tecnologias Inalambricas/tree/main/NFC RFID COM APP](https://github.com/AresGodKiller/Tecnologias%20Inalambricas/tree/main/NFC%20RFID%20COM%20APP)

Video de demostración:

https://youtube.com/shorts/i_WNIqsTOLc
